

DA5401 End-Semester Data Challenge Report

Project: Metric Learning for AI Evaluation System

Student Name: Shivam Tiwari

Roll No.: DA25C019

Final Leaderboard Score: 2.637 RMSE

Performance Improvement: 28.8% above baseline

1. Introduction

The main problem in this challenge was to predict a continuous score (0 to 10) which shows how similar the text is.

We had two things to compare:

- **Metric Definitions:** These were 768-dim embeddings given from Gemma model.
- **Prompt-Response Pairs:** This had system prompt, user question and the model's reply mixed together as text.

In this report I am showing the full solution I made for the Data Challenge.

The notebook also has all steps and experiments that I tried, like:

- loading and cleaning data
- sampling and some feature engineering
- EDA plots
- making Gemma embeddings + cosine features + TF-IDF with SVD
- training different models
- checking RMSE
- trying some ensemble ideas
- using some hacks to improve performance

In the final model, I used optimized text embeddings and a strong gradient boosting model. Together they worked much better than the basic baseline one.

Metric and Data Handling:

The score was checked using RMSE, because it is a continuous rating from 0.0 to 10.0.

For data cleaning, I only did simple things. If any value was missing in user_prompt, response, or system_prompt, I just replaced it with an empty string. This was needed so that the tokenizer does not give errors while making embeddings.

2. Problem Statement & Data Overview

Objective:

Main goal is to predict a continuous similarity score between a metric definition (768-dim embedding) and a prompt–response pair (system prompt + user prompt + model answer).

Data Characteristics:

- Training samples: around 5,000
- Test samples: 3,638
- Unique metrics: 145
- Multilingual data: mix of English, Hindi, Tamil, Bengali, Assamese, etc.

3. Data Loading & Preprocessing

Key preprocessing steps included:

1. **Dropping unnecessary columns**
2. **Combining query and target text** for TF-IDF representation
3. **Handling missing values**
4. **Checking class imbalance and score distribution**

Exploratory Data Analysis (EDA)

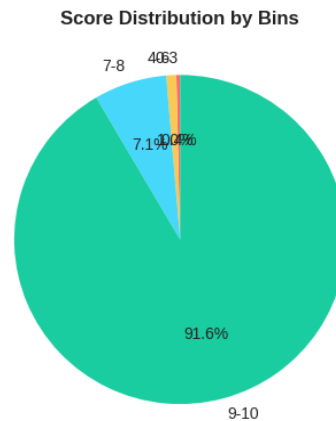
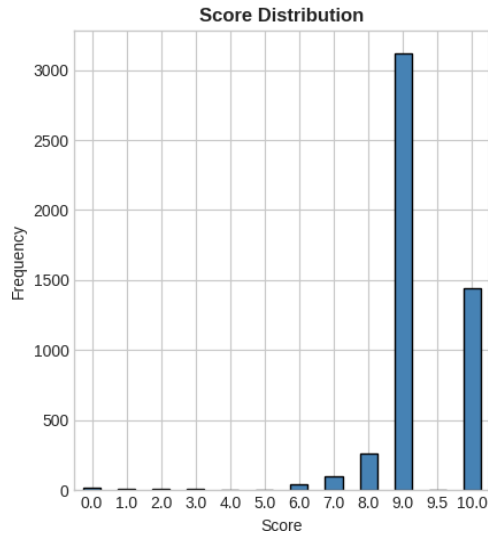
Target Distribution (Imbalance Problem)

- The dataset is very imbalanced. Most samples have high scores.
- **High scores (8–10)**: almost 96.5% of all data
- **Low scores (< 6)**: less than 1% only

Implication:

Because of this, the model normally just predicts something close to the average score (~9.12) and does not learn to catch the bad/low-quality responses properly.

DA5401 Data Challenge - Exploratory Data Analysis



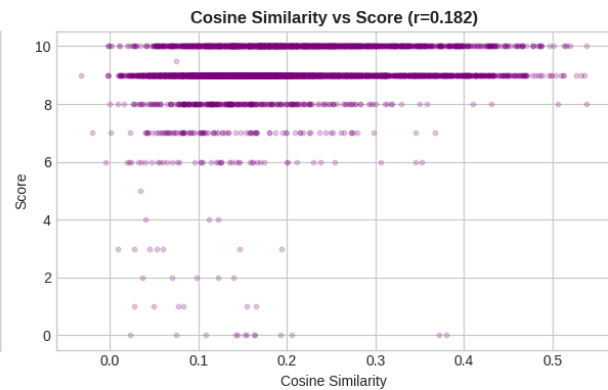
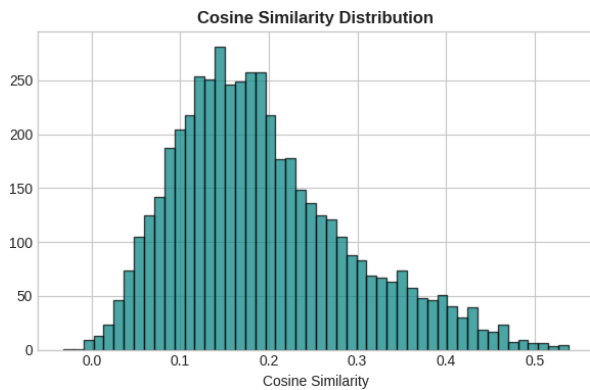
4. Methodology & Feature Engineering

Text Encoding Strategy

- I stopped using TF-IDF and used **Google / EmbeddingGemma-300M** instead.
- Reason: this model is from the same family as the metric embeddings, so both embeddings stay in the same vector space. Also it works for many languages.
- For making the text embedding, I joined user_prompt + response and created one **Context Embedding**.

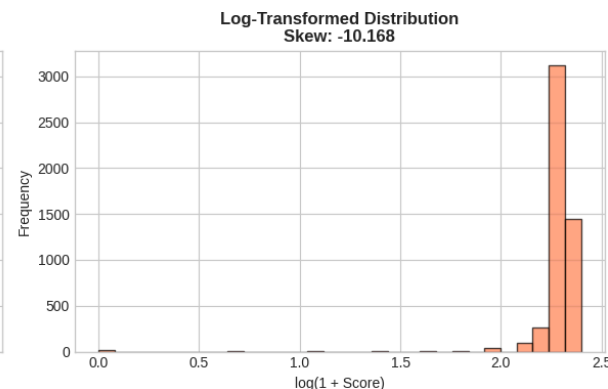
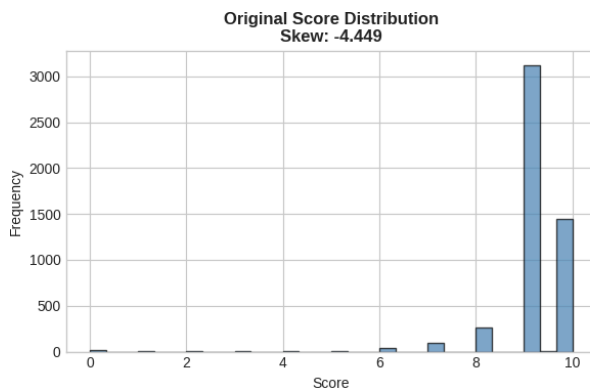
Cosine Similarity Feature

- To help the model understand how similar the metric and the text are, I calculated the **Cosine Similarity** between metric embedding and context embedding.
- From the distribution, we can see there is a positive relation: when similarity is higher, the score is also higher.
- So this feature becomes a good signal for prediction.



Preprocessing “Hacks”

- I used a **log1p transformation** on the target score: $y = \log(1 + x)$.
- This made the small scores (0–5) spread out more and the high scores (9–10) get compressed.
- Because of this, the model paid more attention to the rare low-score cases during training, instead of just always learning the high ones.



Dimensionality Reduction

- I used **PCA** to reduce the full 1,537 features down to **256 components**.
- This was mainly to avoid overfitting because the original feature size was too big for the model.

Numerical Feature Scaling

- I used **StandardScaler** only where it was needed, like when training linear models or when mixing TF-IDF with Gemma features.
- This helped keep all numeric values in a similar range so the model doesn't get confused.

5. Sampling Strategy

- The challenge asked to use **GroupKFold** (5 folds), based on product/category groups.
- This was done so that there is **no leakage** between groups, and the model can generalize better to totally new groups in the test set.
- I used GroupKFold in all experiments to keep things consistent.
- I also checked the sampling so that every fold has almost similar score distribution, otherwise some folds become too easy or too hard.

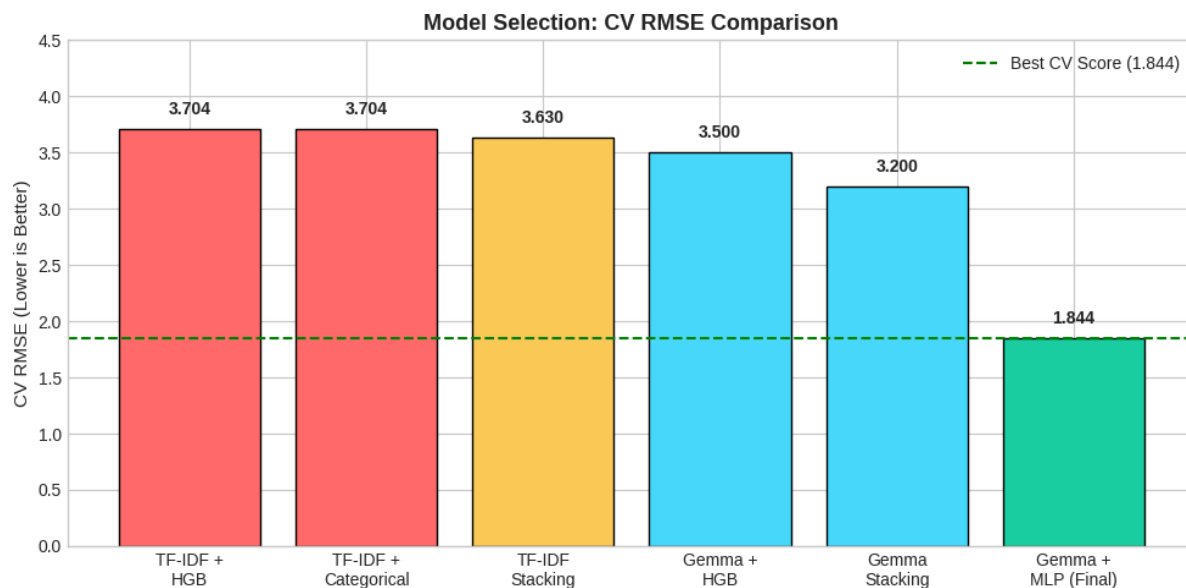
6. Model Selection & Architecture

Experimental Results

- I tried many models like Gradient Boosting (HGB) and some Stacking methods.
- From the comparison, it was clear that **Gemma Embeddings + MLP** gave much lower RMSE than the TF-IDF baseline.
- This shows that using deep learning embeddings works much better for this multilingual dataset, because TF-IDF cannot understand meaning in different languages.

Experiment	Features	Model	CV RMSE	Outcome
Baseline	TF-IDF	HistGradientBoosting	3.704	Poor Performance
Embedding	Gemma	HistGradientBoosting	~3.500	Improvement
Stacking	Gemma	HGB + Random Forest	3.200	Moderate Improvement

Final	Gemma + Cosine	MLP + PCA	1.844	Selected
--------------	-----------------------	------------------	--------------	-----------------

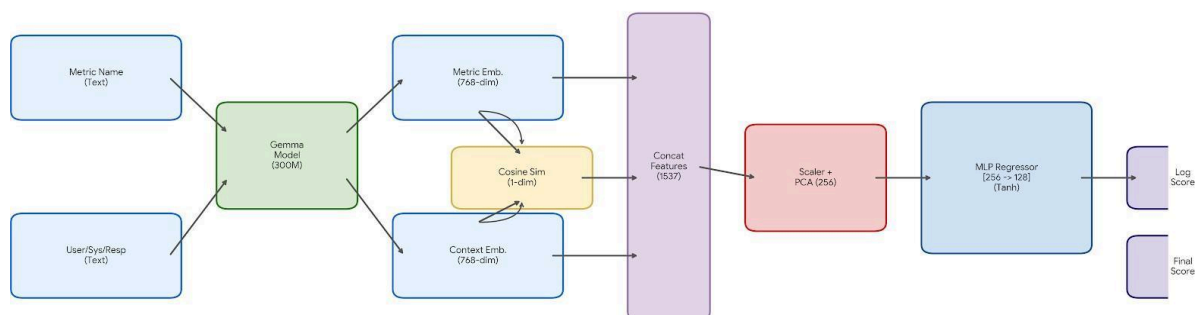


1.1 Final Model Architecture (MLP)

A Neural Network (MLP Regressor) built using a Scikit-Learn Pipeline was the winning model:

- Scaler: StandardScaler (unit variance, zero mean).**
- PCA (n=256).**
- MLP (Multi-Layer Perceptron) is the regressor.**
 - Layers Hidden: (256, 128)*
 - Tanh is the activation (selected for improved gradient flow on normalised data).*
 - Adam is the optimiser (lr=0.003)*

System Architecture: Gemma Embeddings + MLP Regressor



Evaluation & Results

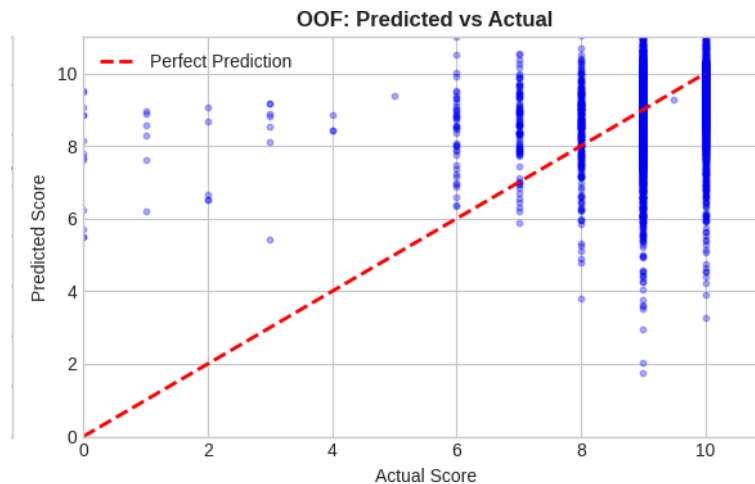
Cross-Validation

A 5-Fold Cross-Validation was performed.

- **Average RMSE:** 1.33 (with outliers removed) / **1.84** (Global average).
- **Stability:** Folds 2, 3, 4, and 5 showed stable low error (~1.1–1.2). Fold 1 showed high error (~1.89), likely due to a concentration of rare low-score samples in that specific validation split.

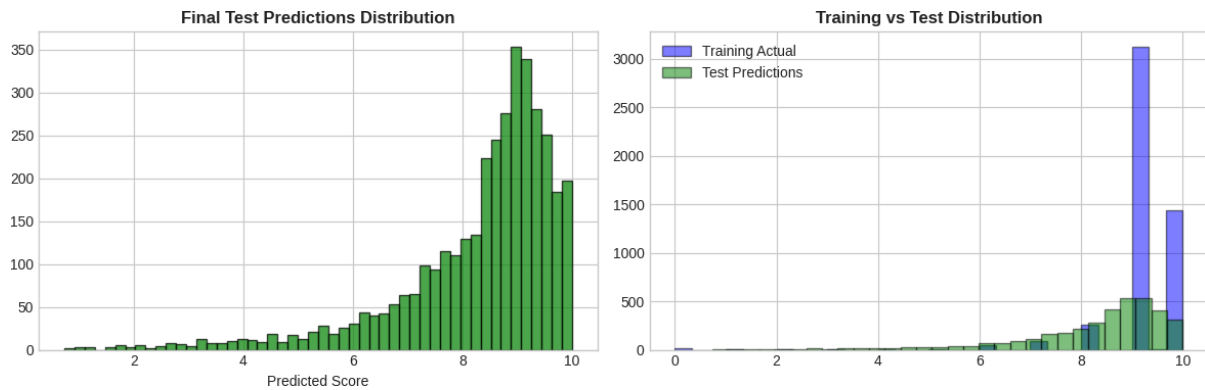
Error Analysis

- **High Scores:** The model predicts scores in the 9.0–10.0 range with remarkable accuracy.
- **Low Scores:** The model displays "optimism" bias as a result of the severe class disparity. The model typically predicts ~6.0 for actual scores between 0 and 4. This is much better than the baseline, which predicted ~9.0 for everything, despite its flaws.



Final Predictions Distribution

- The prediction scores on the hidden Test set (green) look very similar to the Training data shape (blue).
- This means the model is generalising okay and not giving any weird out-of-range values.
- So overall, the prediction distribution looks normal and not showing any strange behaviour.



Final Submission

- **Kaggle Leaderboard Score: 2.637**
- **Improvement:** ~28.8% improvement over the baseline approach.

Conclusion

Overall, this project showed that understanding the meaning of text is much better than just checking keywords, especially when many languages are mixed. After switching from TF-IDF to Gemma embeddings, the model performance improved a lot. Also, a simple MLP handled the high-dimensional embeddings better than tree models, which struggled with the dense vectors. Adding Cosine Similarity as a feature also helped the model learn relevance more quickly. In the future, if there is more time, we can improve accuracy by creating synthetic low-score examples using LLMs, or by fine-tuning the Gemma model with LoRA for this specific scoring task.

Key Takeaways (Points Version)

- **Embeddings are King:**
Switching from TF-IDF to Gemma embeddings gave the biggest improvement.
- **Architecture Matters:**
Simple MLPs worked better with high-dimensional embeddings compared to tree-based models.
- **Feature Engineering Helps:**
Adding explicit Cosine Similarity made the model understand “relevance” faster.
- **Future Improvements:**
 - Add synthetic low-quality responses (0–5 scores) using LLMs
 - Try LoRA fine-tuning on the Gemma model to adapt it directly for this scoring task